

系统架构设计师

架构知识2

姜美荣



目录

直播内容：架构知识

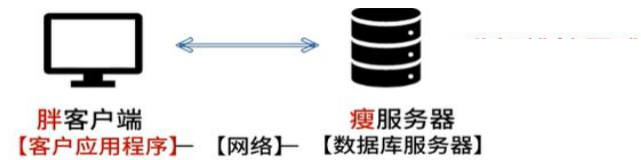
1层次型架构相关案例☆☆☆☆☆

- 表现层设计
- 业务逻辑层组件设计
- 数据访问层设计
- 物联网架构设计

2面向服务的架构相关案例☆☆☆☆☆

- SOA相关概念
- SOA实现模式
- SOA关键技术
- SOA案例

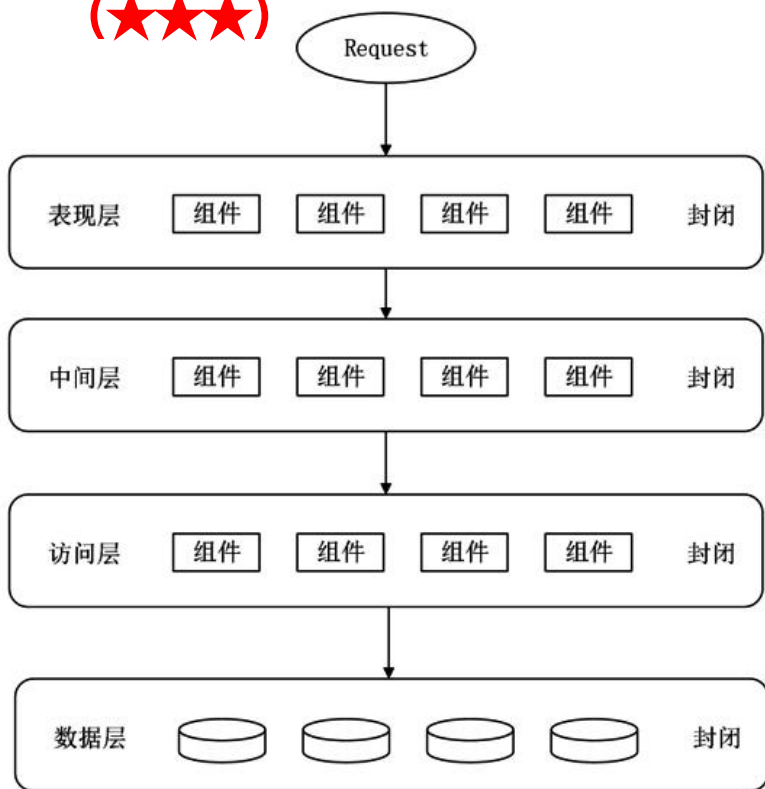
层次式体系结构设计

层次架构	结构	特点
两层C/S	 胖客户端 (业务逻辑) 服务器 (数据处理) 【客户应用程序】—【网络】—【数据库服务器】	升级维护困难
三层C/S	 瘦客户端 (界面输入输出) 应用服务器 (业务逻辑) 服务器 (数据库) 【表示层】—【功能层】—【数据层】	业务逻辑变化不再需要更新客户端
B/S架构	 浏览器 Web应用服务器 (业务逻辑) 服务器 (数据库) 零客户端 (Web浏览器/表示层) WEB应用服务器 (功能层/数据访问层) 数据库服务器 (数据层)	完全不需要客户端 升级维护容易
RIA富联网技术	 客户端 (Ajax、Flex、小程序、HTML5) 等 【胖】客户端 → 【零】客户端 → 【富】客户端 临时下载客户端	临时下载客户端 反应速度快、易于传播、交互性强

层次式体系结构设计

层次式体系结构设计将系统组成一个层次结构，每一层为上层服务，并作为下层客户。内部的层接口只对相邻的层可见，**每一层最多只影响两层，只要给相邻层提供相同的接口**，允许每层用不同的方法实现，为软件重用提供了强大的支持。

层次式架构也称N层架构模式，分成**表现层(展示层)、中间层(业务层)、数据访问层(持久层)和数据层**。
(★★★★)



MVC---MVP---MVVM

ORM

但设计时要注意以下两点：(★)

(1) 要注意**污水池反模式**

污水池反模式，就是请求流简单地穿过几个层，每层里面基本没有做任何业务逻辑，或者做了很少的业务逻辑。如果请求超过20%，则应该考虑让一些层变成开放的。

(2) 需要考虑的是**分层架构可能会让你的应用变得庞大**。

即使你的表现层和中间层可以独立发布，但它的确会带来一些潜在的问题，比如：分布模式复杂、健壮性下降、可靠性和性能的不足，以及代码规模的膨胀等。

分层架构的一个特性就是**关注分离**。(★)

MVC架构风格

MVC模式

MVC 强制地把一个应用的输入、处理、输出流程按照**视图**、**控制**、**模型**的方式进行分离，形成了控制器、模型、视图三个核心模块。

(1)视图 (View)：用户看到并与之交互的界面。视图向用户显示相关的数据，并接收用户输入的数据，但是它并不进行任何实际的业务处理。

(2)控制器(Controller)

该部分是用户界面与模型的接口。解释来自于视图的输入，将其解释成为系统能够理解的对象，同时它也识别用户动作，并将其解释为对模型特定方法的调用；另一方面，它处理来自于模型的事件和模型逻辑执行的结果，调用适当的视图为用户提供反馈。

(3)模型(Model)

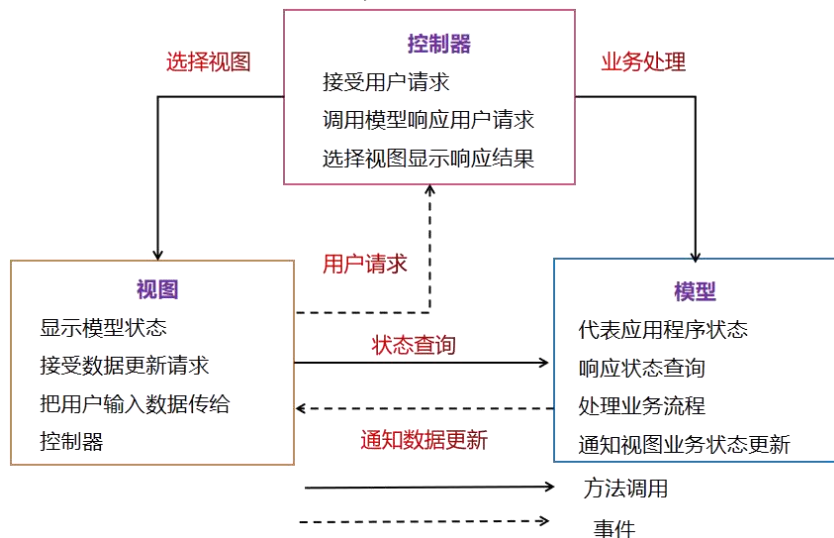
应用程序的主体部分。模型表示**业务数据和业务逻辑**。**一个模型能为多个视图提供数据**。同一个模型可以被多个视图重用，提高应用的可重用性。

使用MVC 模式来设计表现层，可以有以下的优点。

- 1、允许多种用户界面的扩展
- 2、易于维护
- 3、功能强大的用户界面。
- 4、增加应用的可拓展性、强壮性、灵活性。

J2EE体系结构中：

- 1、视图：JSP
- 2、控制：Servlet
- 3、模型：Entity Bean、Session Bean



MVP架构风格

MVP模式

MVP模式中Model提供数据，**View负责显示，Presenter/Controller负责逻辑的处理。** (★★)

MVP与MVC 有一些显著的区别，MVC模式中允许 View 和 Model直接进行"交流"，在MVP 模式中是不允许的。在MVP 中 View 并不直接使用 Model，它们之间的通信是通过 Presenter (MVC中的Controller)来进行的，所有的交互都发生在 Presenter 内部，而在MVC中View会直接从Model 中读取数据而不是通过 Controller。

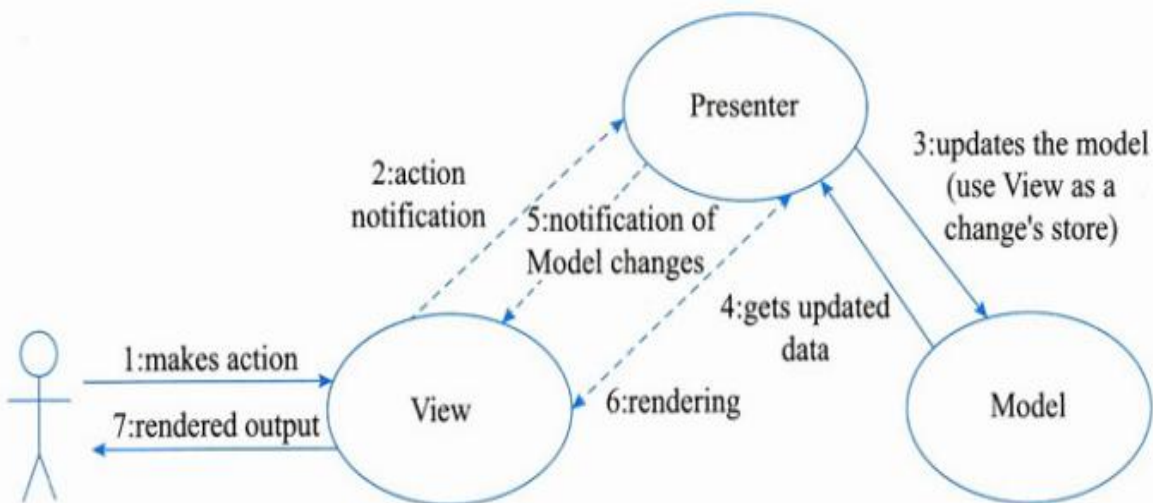


图 13-3 MVP 设计模式

使用MVP模式来设计表现层，可以有以下的优点。

- (1) **模型与视图完全分离**，可以修改视图而不影响模型。
 - (2) 可以更高效地使用模型，**因为所有的交互都发生在一个地方：Presenter 内部。**
 - (3) **可以将一个Presenter 用于多个视图**，而不需要改变Presenter 的逻辑。这个特性非常的有用，因为视图的变化总是比模型的变化频繁。
 - (4) 如果把逻辑放在Presenter 中，就可以脱离用户接口来测试这些逻辑（单元测试）。
- 目前，MVP 模式被更多地用在 Android 开发当中。

MVVM架构风格

MVVM模式 (★★)

MVVM模式正是为**解决MVP中UI种类变多**，接口也会不断增加的问题而提出的。MVVM模式全称是模型-视图-视图模型 (Model-View-ViewModel)，它和MVC、MVP类似，主要目的都是为了实现视图和模型的分离，不同的是 MVVM 中，View 与 Model 的交互通过 ViewModel来实现。ViewModel是MVVM的核心，它通过**DataBinding**实现 View 与Model之间的**双向绑定**，其内容包括数据状态处理、数据绑定及数据转换。例如，View 中某处的状态和Model中某部分数据绑定在一起，这部分数据一旦变化将会反映到 View 层。而这个机制通过 ViewModel来实现。

ViewModel，即视图模型，是一个专门用于数据转换的控制器，它可以把对象信息转换为视图信息，将命令从视图携带到对象。View和ViewModel之间使用DataBinding及其事件进行通信。View的用户接口事件仍然由 View自身处理，并把相关事件映射到ViewModel，以实现View中的对象与视图模型内容的同步，且可通过**双向数据绑定**进行更新。

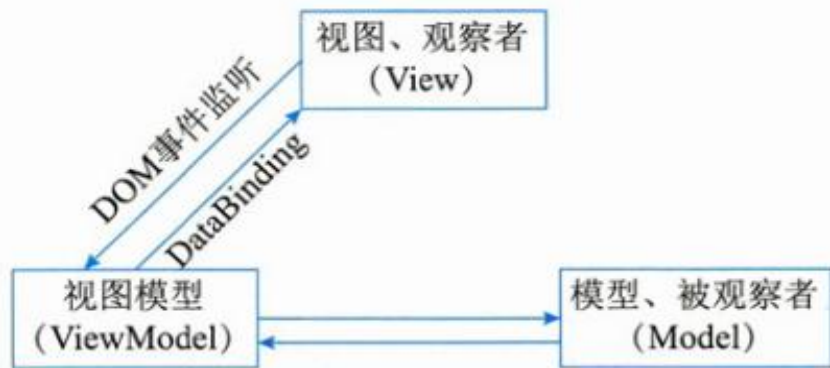


图 13-4 MVVM 设计模式

业务逻辑层框架

业务逻辑层框架。**业务框架位于系统架构的中间层**，是实现系统功能的核心组件。采用**容器**的形式，便于系统功能的开发、代码重用和管理。在业务容器中，业务逻辑是按照 Domain Model—Service—Control 思想来实现的。

其中：

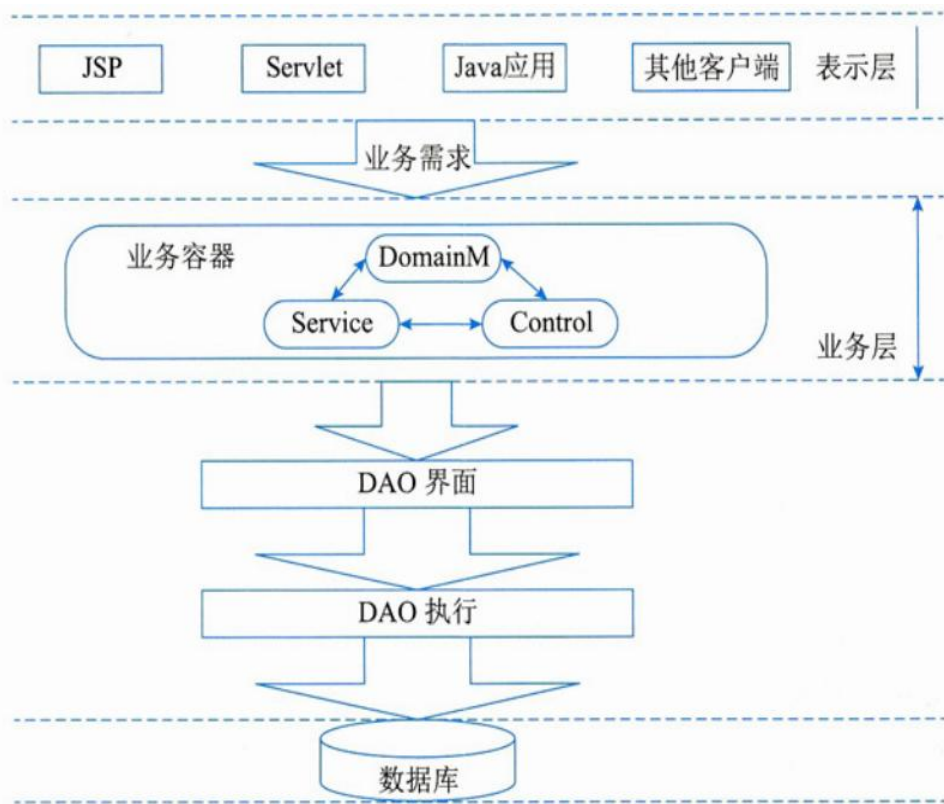


图 13-10 业务框架在整个系统架构中的位置

业务层采用业务容器的方式存在于整个系统当中，采用此方式可以大大降低业务层和相邻各层的耦合，表示层代码只需要将业务参数传递给业务容器，而不需要业务层多余的干预。如此一来，可以有效地防止业务层代码渗透到表示层。

在业务容器中，业务逻辑是按照**Domain Model—Service—Control**思想来实现的。（★★★★）

(1)Domain Model是领域层业务对象，它仅仅包含业务相关的属性。

(2)**Service**是业务过程实现的组成部分，是应用程序的不同功能单元，通过在这些服务之间定义良好的接口和契约联系起来。

(3)**Control**服务控制器，是服务之间的纽带，不同服务之间的切换就是通过它来实现的。

数据访问层设计

5种数据访问模式

模式	特点
在线访问	每个数据库操作都会通过数据库连接不断地与后台的数据源进行交互
Data Access Object	底层数据访问操作 与 高层业务逻辑 分离
Data Transfer Object	跨不同的进程或是网络的边界来传输数据
离线数据模式	从数据源获取数据后，存放到本地处理
对象 / 关系映射	应用系统中的对象与数据库中的数据表形成映射关系

数据访问层设计

对象/关系映射(O/R Mapping) (★★)

对象/关系映射的基本思想来源于这样一种现实: **多数应用的数据都是存在关系型数据库中**, 而这些应用程序中的数据在开发或是运行时是以对象的形式组织起来的, 那么**对象/关系映射**就提供了这样一种工具或平台, 能够将应用程序中的数据转换成关系型数据库中的记录, 或是将关系数据库中的记录转换成应用程序中的代码便于操作的对象。

面向对象	关系数据库
类(class)	数据库的表(table)
对象(object)	记录(record,行数据)
对象的属性(attribute)	字段(field)

技术实现	Hibernate	MyBatis(iBatis)
简单对比	强大, 复杂, 间接, SQL无关	小巧, 简单, 直接, SQL相关
可移植性	好(不关心具体数据库)	差(根据数据库SQL编写)
复杂多表关联	不支持	支持

物联网架构

物联网架构

应用层	解决信息处理和人机交互问题 应用服务 智能终端
平台层	操作系统软件开发设备管理平台连接管理平台
网络层	传递信息和处理信息 网络 通信标准/协议
感知层	解决数据获取问题 传感器 芯片 通信模组

■ 物联网架构

物联网架构

MQTT是一种基于发布/订阅模式的轻量级物联网通信协议。它专门为低带宽、高延迟、不稳定的网络环境设计。

MQTT的QoS一共定义了3个级别分别是：QoS 0、QoS 1、QoS 2。

MQTT协议

相当于互联网的HTTP协议

MQTT (Message Queuing Telemetry Transport, 消息队列遥测传输)

是一种轻量级的、开放的、**基于发布/订阅模式的消息传输协议**，适用于**物联网和传感器网络**等低带宽、不稳定网络环境下的通信需求。特点如下：

- 发布/订阅模式
- 轻量级和节能：适合在带宽和设备资源有限的环境下使用；
- QoS级别：提供3种服务质量（QoS）级别；
- 保持连接：支持保持长时间的连接；
- 安全性支持：支持基于TLS/SSL的加密传输。

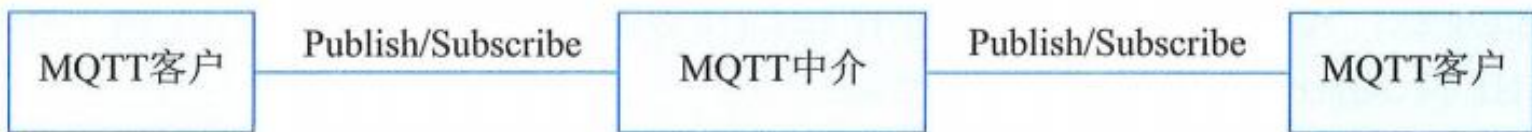
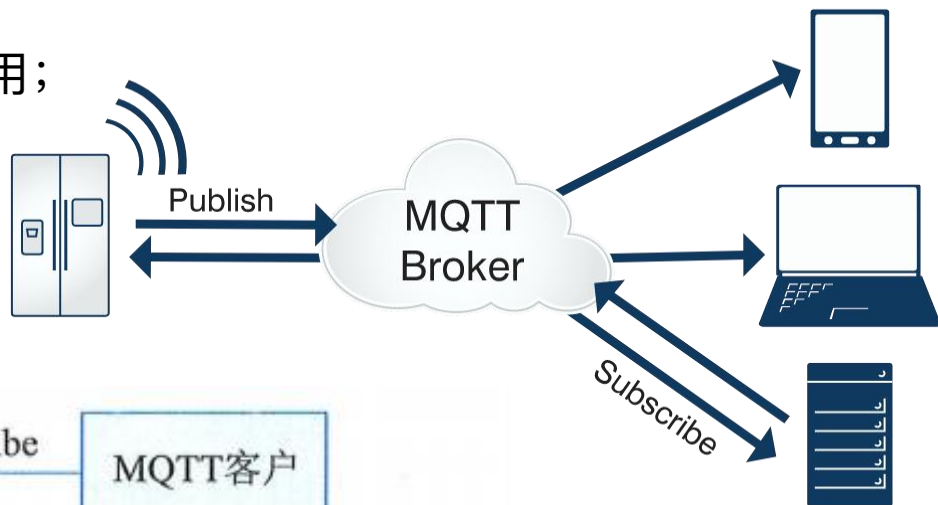


图 21-7 MQTT 基本通信模型

MQTT协议

QoS级别：提供3种服务质量（QoS）级别；

MQTT支持三种QoS级别：QoS0、QoS1和QoS2。

- QoS0是“最多一次”(atmost once)消息传送级别，即消息允许丢失，不进行消息传送确认；
- QoS1是“至少一次”(at least once)消息传送级别，即消息不能丢失，但可允许多次交付；
- QoS2是“仅有一次”(exactlyonce)消息传送级别，即消息不能丢失，也不允许多次交付。

消息传送QoS级别越高，带来的消息传送延迟和网络流量增大。因此，应根据实际传送消息重要程度选择适合的QoS等级。

MQTT提供三类消息持久性选项

- 非持久性(Non-persistent):非持久性选项，即消息不在系统中缓存。此选项适用于非关键类消息传递，或遗失消息容易再发生的场景。
- 队列型持久性(Queued persistent):队列型持久性选项，即在消息无法通知到订阅者时进行队列缓存，以便随后再通知。此选项适用于订阅者不经常在线，或当订阅者可达后需要接收其离线时已发送的所有订阅消息的情况。
- 确认型持久性(Persistent with acknowledgment):确认型持久性选项，即发送给订阅者的消息需得到其确认。此选项适用于需要消息订阅者确认被发送的订阅消息已接收的情形。

MQTT协议

MQ (消息队列):

定义: MQ 是一种应用程序对应用程序的通信方法。应用程序通过写入和读取进出队列中的消息来进行通信, 而不需要建立专门的连接。

用途: 主要用于解决分布式系统的异步处理、负载均衡等问题。

应用场景: 更适用于企业级别的应用集成和服务总线, 常用于需要复杂事务管理和安全性的场合。

MQTT (Message Queuing Telemetry Transport):

定义: MQTT 是一种基于发布/订阅模式的轻量级通讯协议, 主要用于物联网场景下的设备间通信。

特点: 极少的代码和有限的带宽需求, 适合资源受限环境 (如嵌入式系统或传感器)。

应用场景: 主要在物联网领域广泛应用, 例如智能家电、智慧城市等, 强调的是低功耗和可靠性

MQTT协议 VS HTTP协议

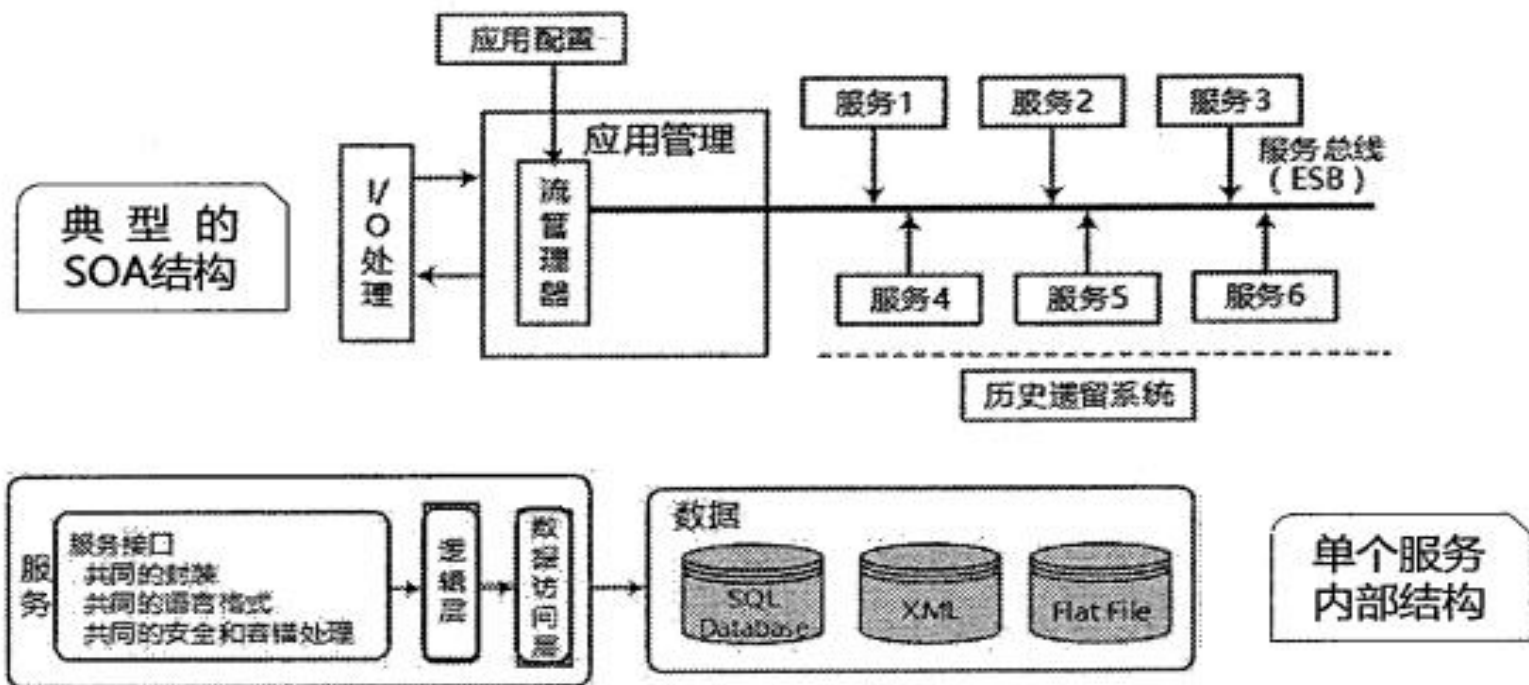
对比维度	HTTP（超文本传输协议）	MQTT（消息队列遥测传输协议）
定义	基于请求 - 响应模式的应用层协议，用于客户端与服务器之间传输超文本数据	基于发布 / 订阅模式的轻量级消息协议，专为低带宽、不稳定网络环境设计
通信模式	请求 - 响应（一对一）	发布 / 订阅（一对多，解耦生产者与消费者）
连接方式	短连接为主（HTTP/1.1 支持长连接复用）	长连接（基于 TCP，保持持久会话）
消息模型	同步阻塞，请求必须等待响应	异步非阻塞，消息推送实时到达
协议开销	较大，请求头 / 响应头包含大量元数据（通常几百字节到几 KB）	极小，最小报文仅 2 字节，头部开销极低
QoS 等级	无内置 QoS 保证，依赖 TCP 可靠传输	支持 3 级 QoS：
		· QoS 0：最多一次
		· QoS 1：至少一次
		· QoS 2：恰好一次
带宽占用	高，文本格式 + 冗余头部	低，二进制格式 + 精简头部
实时性	较差，需客户端轮询获取新数据	高，服务端主动推送，延迟低
资源消耗	较高，每次请求建立连接消耗资源	极低，适合算力 / 内存受限设备
安全机制	HTTPS（TLS 加密）、OAuth、JWT 等	TLS 加密、用户名 / 密码认证、客户端证书
典型应用场景	· Web 网页浏览	· 物联网（IoT）设备通信
	· REST API 接口调用	· 智能家居控制
	· 文件上传下载	· 工业传感器数据采集
	· 移动端 App 数据请求	· 车联网消息推送
	· 电商、社交等互联网应用	· 即时通讯（IM）
		· 低功耗设备远程监控
适用设备	PC、服务器、智能手机等算力充足的设备	传感器、嵌入式设备、MCU 等资源受限设备
网络环境要求	稳定、高带宽网络	低带宽、高延迟、不稳定网络（如移动网络、卫星通信）
数据格式	文本为主（JSON、XML、HTML）	二进制 Payload，格式自定义
代表实现 / 框架	Nginx、Apache、Node.js Express、Spring Boot	EMQX、Mosquitto、HiveMQ、Paho

SOA 的相关概念

从软件的基本原理定义，可以认为SOA是一个组件模型，它将**应用程序的不同功能单元(称为服务)**通过这些服务之间定义良好的接口和契约联系起来。接口是采用中立的方式进行定义的，它应该独立于实现服务的硬件平台、操作系统和编程语言。

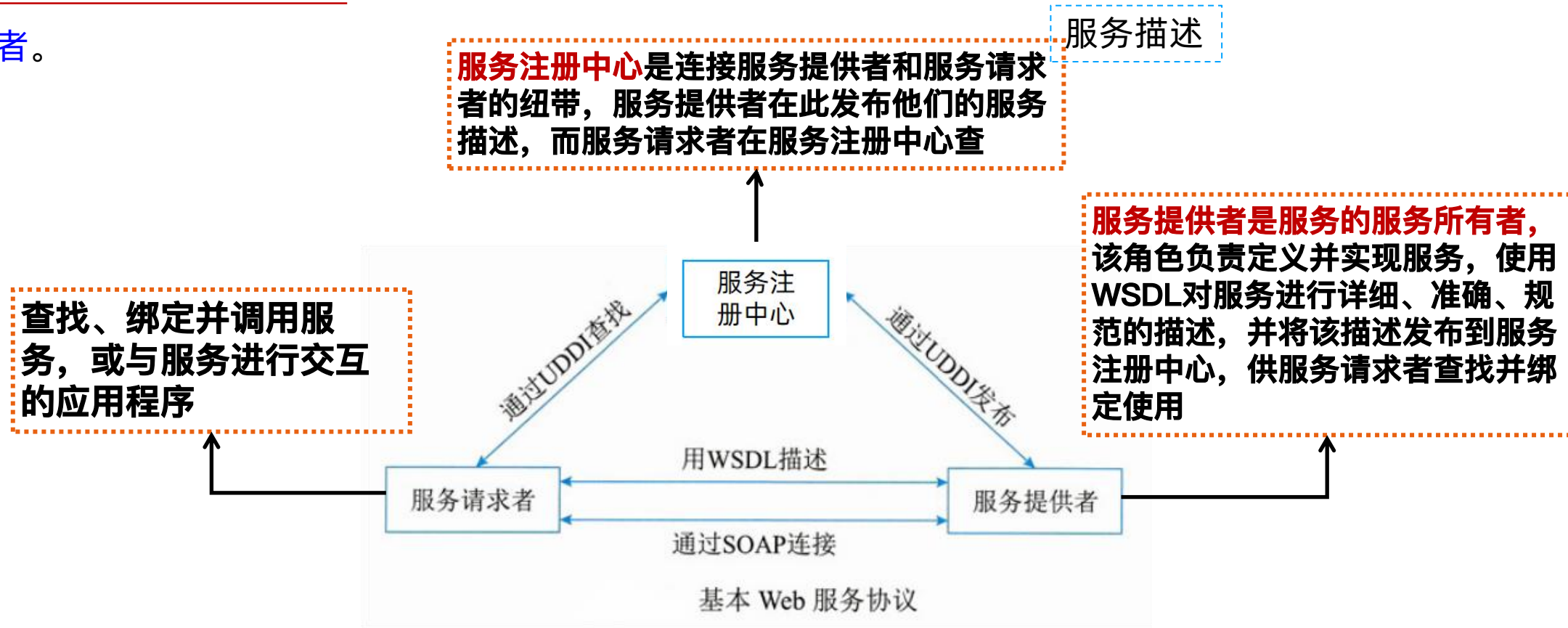
SOA是一种**粗粒度、松耦合**服务架构，**标准化**接口进行通信，不涉及底层编程接口和通信模型。

业务流程是指为了实现某种业务目的行为所进行的流程或一系列动作。



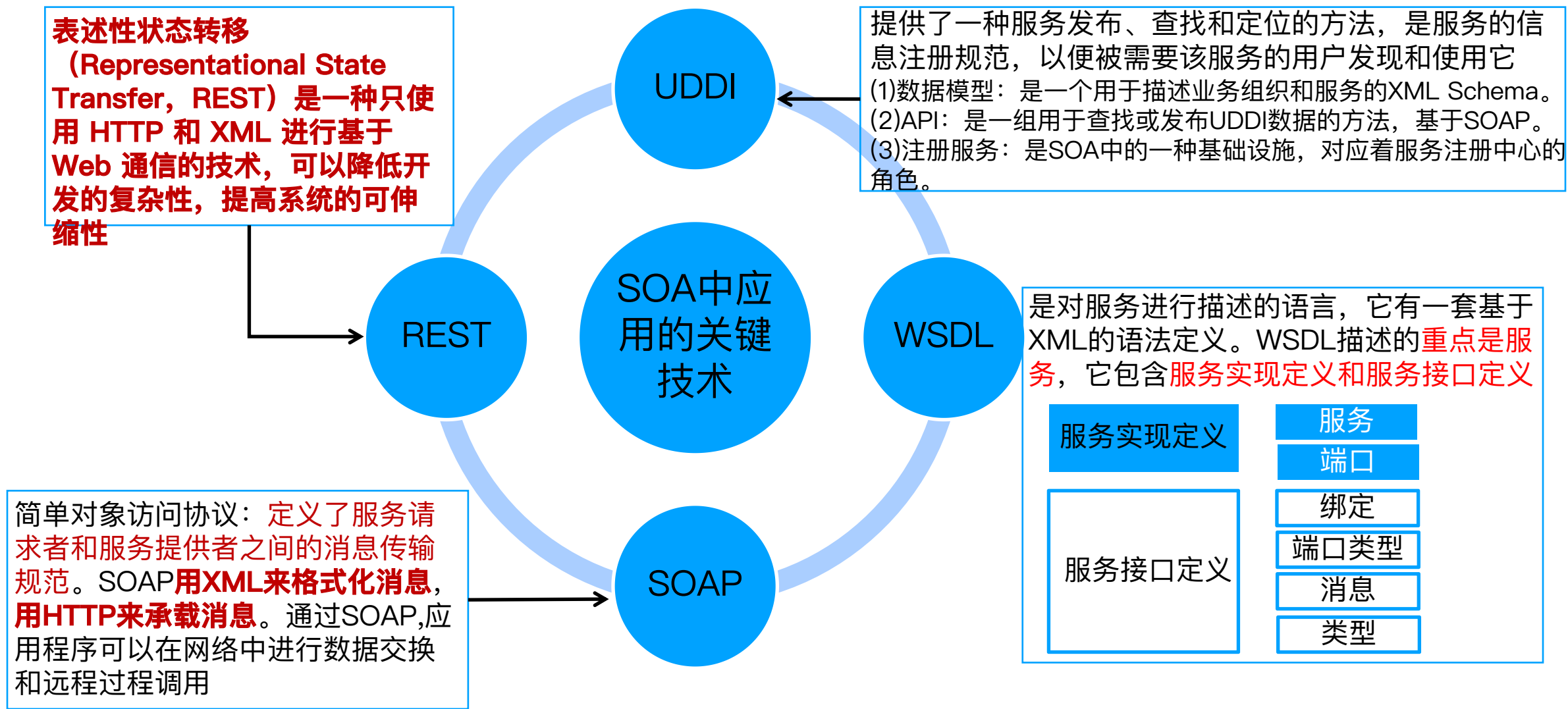
SOA 的实现模式-Web Service

一、**WEB Service**: 服务提供者、服务注册中心（中介，提供交易平台，可有可无）、服务请求者。



WSDL 就是 WebService 接口对应的 WSDL 文件，该文件通过 xml 格式说明如何调用，可以看作 WebService 的接口文档 (使用说明书)。

面向服务的架构风格-SOA关键技术



■ 表述性状态转移

REST(Representational State Transfer,表述性状态转移)是一种通常使用HTTP和XML进行基于Web通信的技术，可以降低开发的复杂性，提高系统的可伸缩性。

REST的5个原则 (★★★★)

- (1)网络上的所有事物都被抽象为资源。
- (2)每个资源对应一个唯一的资源标识。
- (3)通过通用的连接件接口对资源进行操作。
- (4)对资源的各种操作不会改变资源标识。
- (5)所有的操作都是无状态的。

REST API

命令

用途

GET

从指定的资源请求数据，只进行**数据检索**，不进行其他操作

POST

将数据发送到服务器进行**创建**，通常用于上传文件或提交表单

PUT

更新目标资源的所有当前表示，使用上传的内容进行**替换**

DELETE

删除指定的资源

HEAD

与GET方法类似，但**只传输状态行和头部信息**

PATCH

对资源进行**部分修改**

SOA 的实现模式-企业服务总线模式

二、企业服务总线模式

企业服务总线 (ESB), 提供一种标准的软件底层架构, 各种程序组件能够以服务单元的方式“插入”到该平台上运行, 并且组件之间能够以标准的消息通信方式来进行交互。

ESB 本质上是以中间件形式支持服务单元之间进行交互的软件平台。各种程序组件以标准的方式连接在该“总线”上, 并且组件之间能够以格式统一的消息通信的方式来进行交互。

服务总线事实上实现了组件和应用系统的位置透明和协议透明。技术人员可以通过开发符合 ESB 标准的组件(适配器)将外部应用连接至服务总线, 实现与其他系统的互操作。同时, ESB 以中间件的方式, 提供服务容错、负载均衡、QoS 保障和可管理功能。

ESB的核心功能如下:

- (1)提供**位置透明的消息路由和寻址服务**。
- (2)提供**服务注册和命名的管理功能**。
- (3)支持**多种消息传递范型**(如请求/响应、发布/订阅等)
- (4)支持**多种可以广泛使用的传输协议**。
- (5)支持**多种数据格式及其相互转换**。
- (6)提供**日志和监控功能**。



SOA 的设计模式-微服务模式

三、微服务模式

优点	说明	缺点
复杂应用解耦	小服务(且专注于做一件事情) 化整为零, 易于小团队开发	微服务架构面临的问题与挑战。 (1) 服务发现与服务调用链跟踪变得困难。 (2) 很难实现传统数据库的强一致性, 转而追求最终一致性。 (3) 测试复杂性 (服务间依赖测试) (4) 运维更加复杂 (5) 并非所有系统都可拆分为微服务 (6) 微服务的部署也更加困难
独立	独立开发 独立测试及独立部署(简单部署) 独立运行(每个服务独立在其独立进程中)	
技术选型灵活	支持异构(如: 每个服务使用不同数据库)	
容错	故障被隔离在单个服务中, 通过重试、 平稳退化等机制实现应用层容错	【微服务与SOA的区别】 微服务比SOA更精细, 可以独立进程方式存在。 微服务接口更通用化, 如用HTTP、RESTful, 各种终端都可调用, 语言无关, 平台无关。 更倾向于分布式部署, 互联网场景更适合。
松耦合, 易扩展	可根据需求独立扩展	

SOA 的设计模式-微服务模式

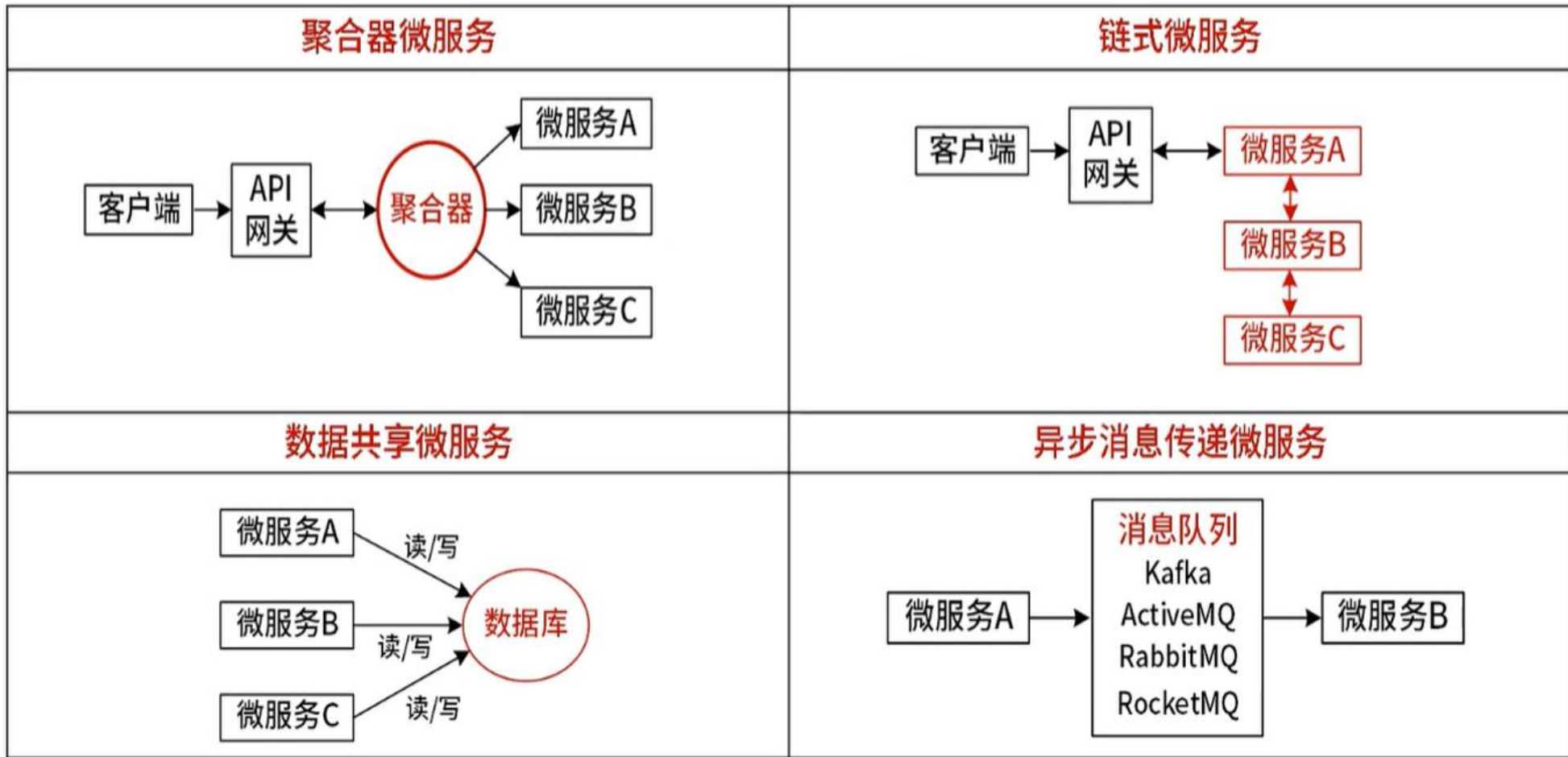
微服务架构模式方案。（★★）

主要包括：

- (1) **聚合器微服务**：聚合器充当流程指挥者，调用多个微服务实现系统应用程序所需功能。
- (2) **链式微服务**：客户端或服务在收到请求后，会发生多个服务间的嵌套递归调用，返回经过合并处理的响应。
- (3) **数据共享微服务**：该模式适用于在单体架构应用到微服务架构的过渡阶段，服务之间允许存在强耦合关系，例如存在多个微服务共享缓存与数据库存储的现象。
- (4) **异步消息传递微服务**：对于一些不必要以同步方式运行的业务逻辑，可以**使用消息队列**代替 REST 实现请求、响应，加快服务调用的响应速度。

SOA 的设计模式-微服务模式

【微服务架构模式方案】



■ 典型例题（架构2025.11）

试题四（25分）

某传统物流企业计划开发一套新一代物流管理系统，要求支持实时每一件货物追踪、路径优化、多平台订单接入及大数据分析功能。系统需应对高并发订单处理、海量传感器数据实时分析、跨平台（Web/移动端/IoT设备）兼容等挑战。开发过程中，团队面临以下问题：

- 表现层代码紧密结合业务逻辑，维护成本高；
- 持久层需支持复杂查询与高性能数据读写；
- 实时货物数据导致数据库大量写操作，几乎占满了磁盘I/O，延迟明显。

【问题1】（8分）

李工建议通过表现层分层（如MVC或MVVM模式）优化代码结构，请以200字以内的文字比较MVC模式与MVVM模式的优缺点。

MVC的优势：分层简单，各层专注自身的职责，允许多种用户界面的扩展，易于维护；

MVC的缺点：视图层与模型层存在直接耦合，绕过了控制器。

MVVM的优势：通过数据绑定（如ViewModel）实现视图与模型的自动同步，防止了视图层与模型层的直接耦合。

MVVM的缺点：出现问题不容易定位来源。

典型例题

【问题2】（7分）

赵工主张引入持久层框架提升数据持久化效率。若需在Hibernate与MyBatis中选择一个作为持久层框架，请从性能、灵活性、复杂查询支持三个维度阐述选择依据，字数300字以内。

应选择MyBatis作为持久层框架。

理由如下：

- (1) 性能：Hibernate通过全自动ORM映射可能引入额外开销，尤其在复杂查询或大数据量场景下；MyBatis允许手动优化SQL，减少不必要的数据库交互，更适合高并发读写场景。
- (2) 灵活性：MyBatis支持原生SQL和动态SQL，可灵活支持不同数据库特性（如随机数等特殊函数）；Hibernate的HQL虽跨数据库，但复杂业务逻辑较难实现，灵活性较低。
- (3) 复杂查询支持：MyBatis直接编写多表关联、子查询等复杂SQL更直观高效；Hibernate的HQL对复杂查询支持程度有限，在处理多表关联时可能生成不优化的SQL。

技术实现	Hibernate	MyBatis(iBatis)
简单对比	强大，复杂，间接，SQL无关	小巧，简单，直接，SQL相关
可移植性	好(不关心具体数据库)	差(根据数据库SQL编写)
复杂多表关联	不支持	支持

典型例题

【问题3】（10分）

陈工提出采用边缘计算技术实现数据低延迟处理。请比较边缘计算与集中式处理在物流系统中的差异，并填写下表。

特性	边缘计算	集中式处理
数据处理位置	(1)	云端数据中心
延迟表现	低延迟（毫秒级）	(2)
存在问题	(3)	网络带宽和海量数据处理能力
可靠性	较低	(4)
典型适用场景	(5)	需要全局分析的海量数据处理

- (1) 边缘节点
- (2) 较高延迟（秒级）
- (3) 有限的硬件资源和恶劣的部署环境
- (4) 很高
- (5) 实时性要求高的本地决策

典型例题

阅读以下关于 Web 系统设计的叙述，在答题纸上回答问题1至问题3。

【说明】

某银行拟将以分行为主体的银行信息系统，全面整合为由总行统一管理维护的银行信息 系统，实现统一的用户账户管理、转账汇款、自助缴费、理财投资、贷款管理、网上支付、 财务报表分析等业务功能。但是，由于原有以分行为主体的银行信息系统中，多个业务系统 采用异构平台、数据库和中间件，使用的报文交换标准和通信协议也不尽相同，使用传统的 EAI 解决方案根本无法实现新的业务模式下异构系统间灵活的交互和集成。因此，为了以最 小的系统改进整合现有的基于不同技术实现的银行业务系统，该银行拟采用基于ESB的面向服务架构（SOA） 集成方案实现业务整合。

【问题1】（7分）

请分别用200字以内的文字说明什么是面向服务架构（SOA） 以及ESB 在 SOA 中的作用与特点。

典型例题

【问题1】 参考答案

面向服务的架构 (SOA) 是一种粗粒度、松耦合服务架构，服务之间通过简单、精确定义接口进行通信。它可以根据需求通过网络对松散耦合的粗粒度应用组件进行分布式部署、组合和使用。SOA能帮助企业系统架构设计者以更迅速、更可靠、更高重用性设计整个业务系统架构，基于SOA的系统能够更加从容地面对业务的急剧变化。

企业服务总线 (ESB) 是由中间件技术实现的全面支持面向服务架构的基础软件平台，支持异构环境中的服务以及基于消息和事件驱动模式的交互，并且具有适当的服务质量和可管理性。

典型例题

【问题2】 (12分)

基于该信息系统整合的实际需求，项目组完成了基于SOA 的银行信息系统架构设计方案。该系统架构图如图5-1所示。请从(a)~(k) 中选择相应内容填入图5-1的(1)~(6)，补充完善架构设计图。

- (a) 数据层
- (b) 界面层
- (c) 业务层
- (d) bind
- (e) 企业服务总线ESB
- (f) XML
- (g) 安全验证和质量管理
- (h) publish
- (i) UDDI
- (j) 组件层
- (k) BPEL

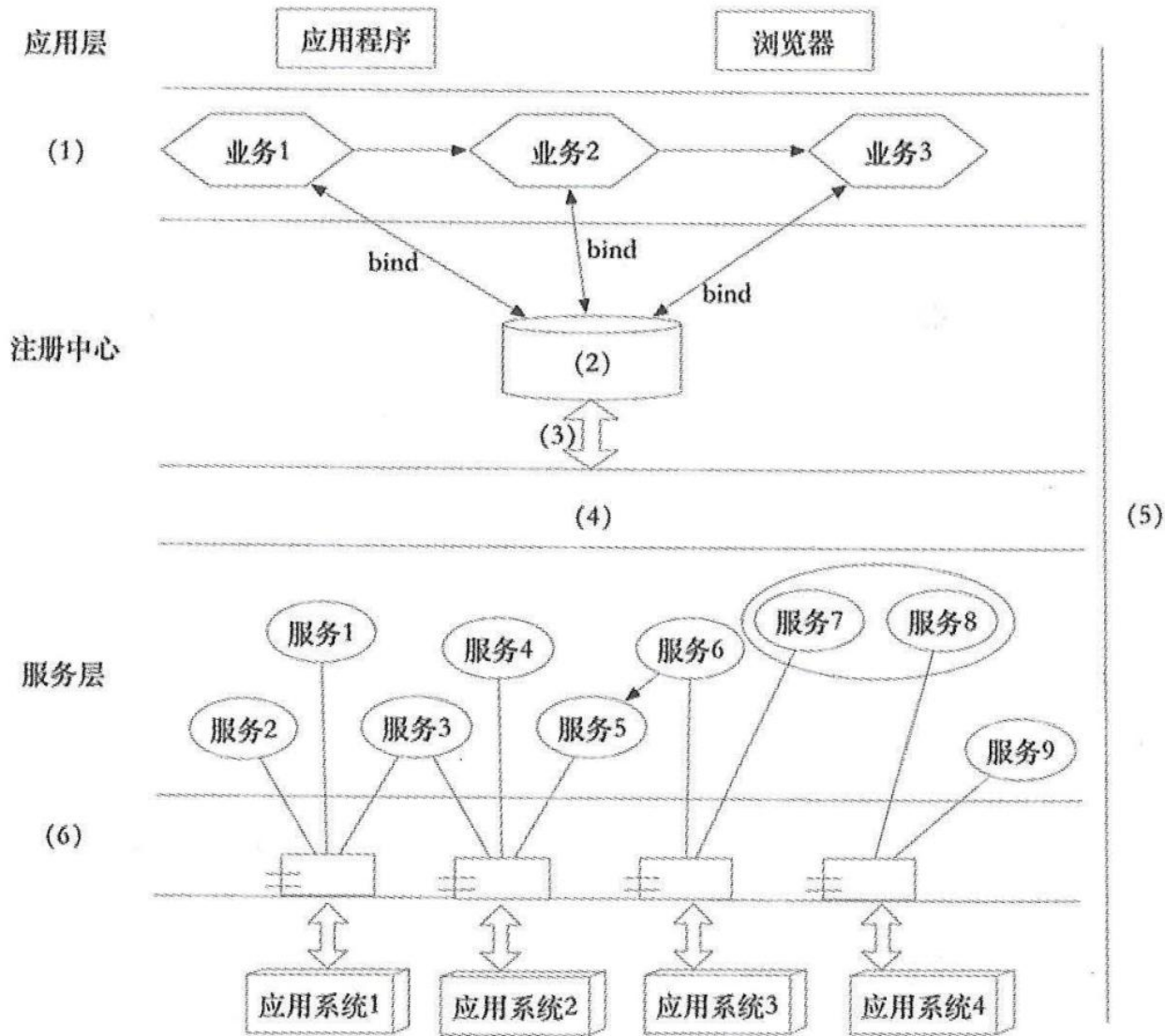


图 5-1 基于 SOA 的银行信息系统架构设计

典型例题

【问题2】 (12分)

基于该信息系统整合的实际需求，项目组完成了基于SOA 的银行信息系统架构设计方案。该系统架构图如图5-1所示。请从(a)~(k) 中选择相应内容填入图5-1的(1)~(6)，补充完善架构设计图。

- (a) 数据层
- (b) 界面层
- (c) 业务层
- (d) bind
- (e) 企业服务总线ESB
- (f) XML
- (g) 安全验证和质量管理
- (h) publish
- (i) UDDI
- (j) 组件层
- (k) BPEL

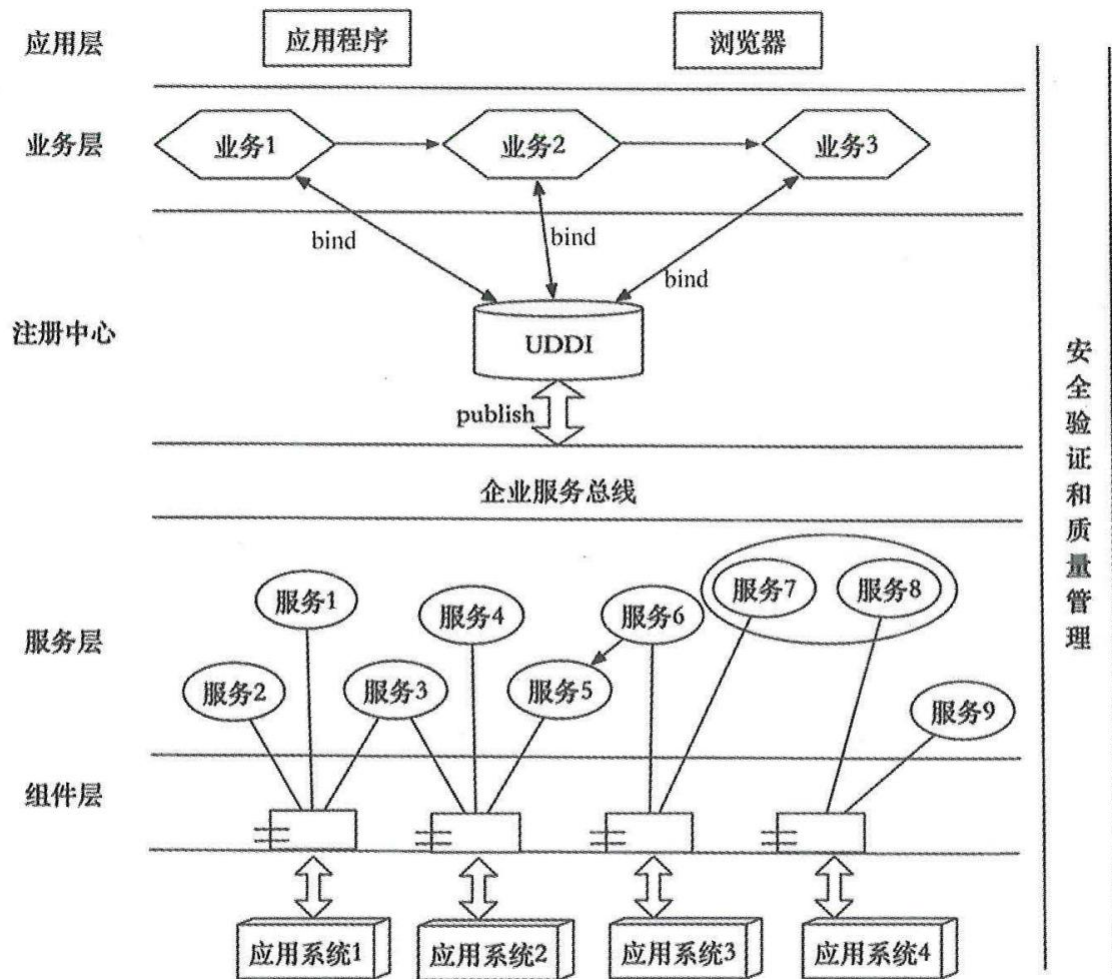


图 5-2 基于 SOA 的银行信息系统完整架构设计

典型例题 (2025系分真题)

案例分析五

阅读以下关于 Web 应用系统的描述，回答问题 1 至问题 3。

【说明】

为了保障市民出行安全，提高出行效率，建设绿色交通，某城市委托某公司开发一个基于 Web 的地铁站智慧管理系统。该系统的主要需求分析如下：

(1) 智慧指挥。对地铁站日常指挥工作提供所需信息，包括车站端和车辆端。其功能有视频巡检、客流热力图、列车位置、摄像头分布、预案管理和执行结果反馈；

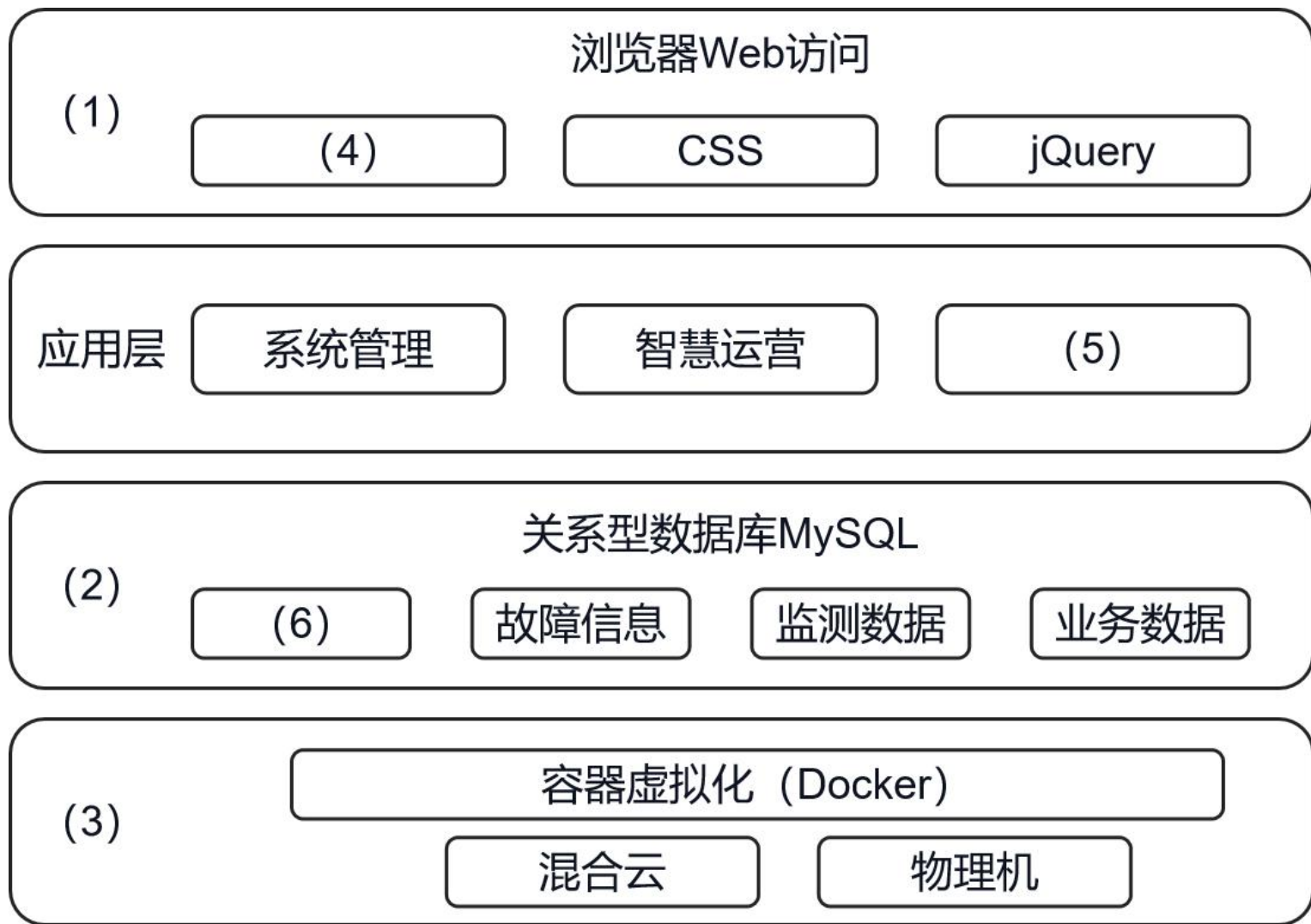
(2) 智慧运营。帮助站内调度人员实现各调度角色之间的调度命令，降低劳动强度，提高劳动效率，包括交接班管理、检修工作管理、工作日志显示、文档管理和事件查询等；

(3) 系统管理。提供了角色管理、部门管理、摄像头管理、岗位管理、用户管理和联动管理等六个功能；

(4) 地铁业务有明显的日常和峰值流量，涉及市民出行安全，应该注意系统和存储的高并发性、可扩展性和安全性。

该公司针对上述需求组建了项目组，经项目组讨论，决定采用 MVC 架构，使用 SSM 框架实现该系统。

典型例题



系统MVC架构图

典型例题

【问题 1】（9 分）

请用 200 字以内文字说明

(1) 什么是 MVC 架构？

(2) 如何利用 SSM 框架来实现 MVC 架构。

参考答案：

(1) MVC架构

模型（Model）：负责管理数据和业务逻辑，直接与数据库交互，处理数据的存储、检索和更新。

视图（View）：用于展示数据和用户界面，仅负责呈现内容，不包含业务逻辑。

控制器（Controller）：作为桥梁连接模型和视图，接收用户输入，调用模型处理数据，并根据结果选择对应的视图进行展示。

(2) 如何利用 SSM 框架来实现 MVC 架构。

SSM 框架由 Spring、Spring MVC 和 MyBatis 三部分组成。

Model（模型）：表示数据层，通常由 MyBatis 负责实现与数据库的交互（持久层）。

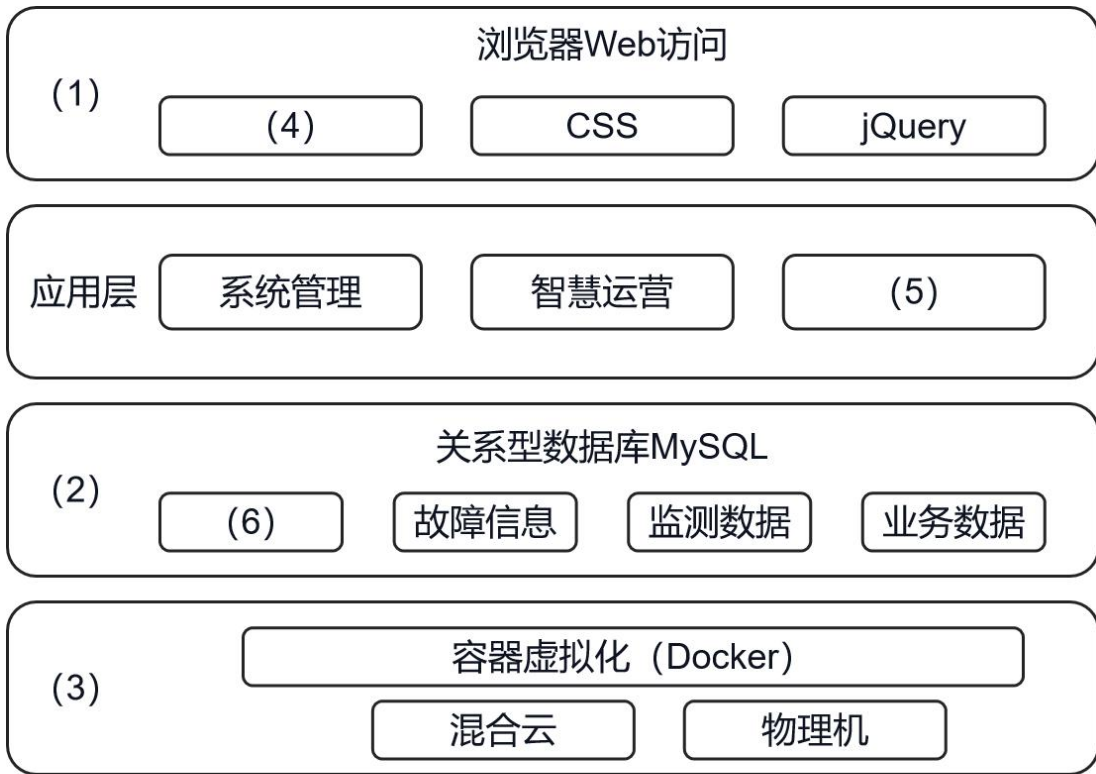
View（视图）：负责数据的展示，通常用 JSP、HTML 或前端框架（如 Vue、Thymeleaf 等）。

Controller（控制器）：负责业务处理和请求分发，通常由 Spring MVC 控制器（如 @Controller 类）实现。

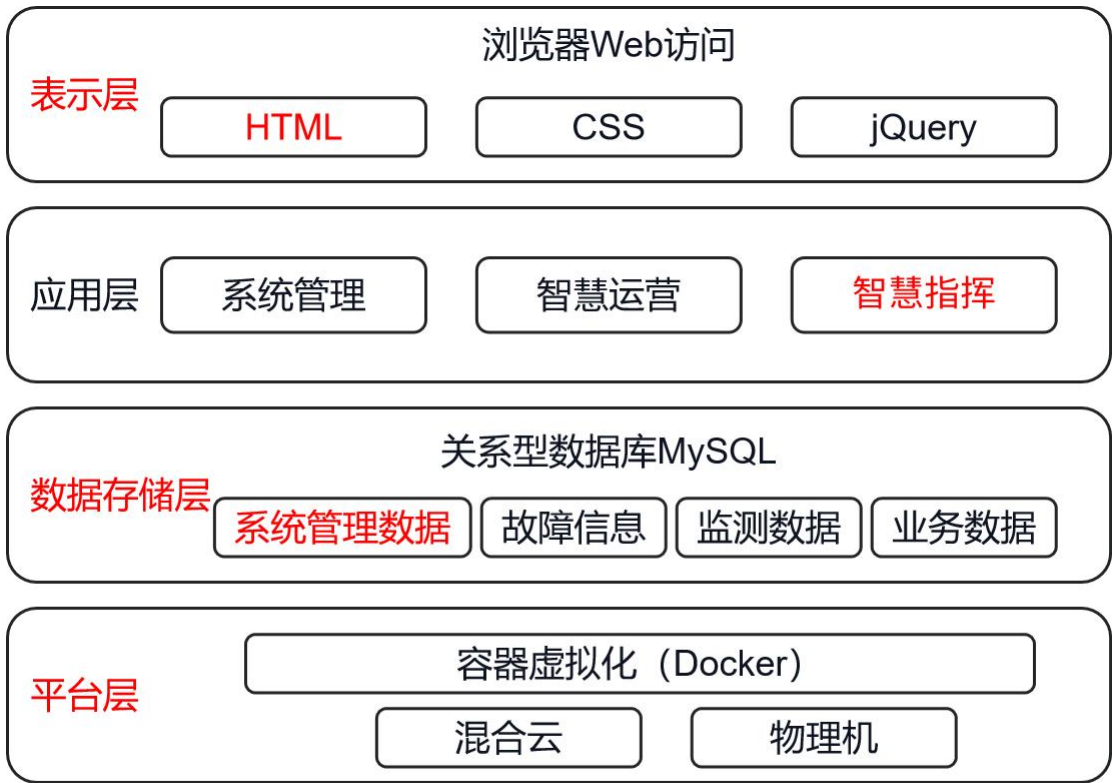
典型例题

【问题 2】 (6 分)

请基于对 SSM 框架的了解，补充完善图 1 的空白处 (1) ~ (6)，完成系统的软件架构图。



系统MVC架构图



系统MVC架构图

典型例题

【问题 3】（10 分）

云存储是一种通过互联网将数据存储在远程服务器上的技术，用户可按需访问和管理数据。项目组在讨论采用哪种云存储时，张工建议使用公有云、李工建议使用私有云，经讨论最终决定使用混合云，请比较公有云、私有云和混合云的区别，填写表 1 中的 (1) ~ (9)，并说明选用混合云的理由。

	公有云	私有云	混合云
所有权	(1)	(4)	(7)
安全性	(2)	(5)	(8)
扩展性	(3)	(6)	(9)

	公有云	私有云	混合云
所有权	公开	单个组织	部分公开、部分私有
安全性	低	高	中等
扩展性	高	低	中等

作业

2025.11 【问题 1】 (7分)

简要说明微服务的概念以及优势与挑战。

作业

【参考答案】

微服务是一种架构风格，将单体应用划分成一组小的服务，服务之间相互协作，实现业务功能每个服务运行在独立的进程中，服务间采用轻量级的通信机制协作(通常是HTTP/JSON)，每个服务围绕业务能力进行构建，并且能够通过自动化机制独立地部署。

1、微服务有以下优势：

- (1) 通过分解巨大单体式应用为多个服务方法解决了复杂性问题。它把庞大的单一模块应用分解为一系列的服务，同时保持总体功能不变，但整体并发却得到极大提升。
- (2) 让每个服务能够独立开发，开发者能够自由选择可行的技术，提供API服务，数据库的结构。
- (3) 微服务架构模式是每个微服务独立的部署。开发者不再需要协调其他服务部署对本服务的影响。这种改变可以加快部署速度。
- (4) 微服务使得每个服务独立扩展。可以根据每个服务的规模来部署满足需求的规模，甚至可以使用更适合于服务资源需求的硬件。

2、微服务架构带来的挑战如下：

- (1) 并非所有的系统都能转成微服务。
- (2) 部署较单体架构更加复杂：系统由众多微服务搭建，每个微服务需要单独部署，从而增加部署的复杂度，容器技术能够解决这一问题。
- (3) 性能问题：由于微服务注重独立性，互相通信时只能通过标准接口，可能产生延迟或调用出错。
- (4) 数据一致性问题：作为分布式部署的微服务，在保持数据一致性方面比传统架构更加困难。

作业

某互联网公司拟开发一款电商移动端应用，该应用需具备界面交互频繁、数据更新实时性要求高、后续迭代维护成本控制严格等特点。技术团队在架构选型阶段，围绕 MVP 和 MVVM 两种架构模式展开讨论，团队成员对两种架构的核心设计思想、组件交互方式、数据流转规则等存在不同理解，同时需要结合业务特点选择适配的架构并阐述技术选型依据。

问题 1（理论知识题10分）

请简述 MVP 和 MVVM 架构的核心设计理念，以及两种架构在核心协调组件上的区别。（10 分）

问题 2（基础应用题8分）

结合题干给出的架构特性选项（A-F），请分别选出属于 MVP 架构的典型特性、属于 MVVM 架构的典型特性。

A.Presenter 手动更新 View

B. View 通过数据绑定自动响应 ViewModel 的变化

C. Model 直接通知 View 刷新

D. MVVM 支持双向数据绑定

E. View 实现接口供 Presenter 调用

F. ViewModel 直接操作 DOMG. View 可直接访问 ModelH. 数据只能从 View 流向 ViewModel

架构模式	数据如何更新?	View 如何与逻辑层交互?
MVP	(1)	(2)
MVVM	(3)	(4)

■ 作业

问题 3（综合分析题7分）

结合该电商移动端应用“界面交互频繁、数据更新实时性高、后续迭代维护成本控制严格”的业务特点，分析该应用更适合采用 MVP 还是 MVVM 架构，并从组件交互效率、开发维护成本、数据流转适配性三个维度详细阐述选型依据。

作业

问题1

核心设计理念

1.MVP：基于 MVC 架构优化而来，核心是引入Presenter作为 View 和 Model 的中间协调层，通过接口化设计实现 View 与 Model 的完全解耦；View 仅负责视图渲染和用户交互，Model 仅处理数据逻辑和业务请求，所有的通信、业务逻辑处理均由 Presenter 完成。（3 分）

2.MVVM：在 MVP 基础上进一步升级，核心是通过数据绑定实现 View 和 ViewModel 的双向联动，让视图与数据完全分离；View 负责展示，ViewModel 封装业务逻辑和数据状态，Model 仍处理底层数据，无需中间层手动协调，数据变化可自动同步至视图，视图交互也可自动更新数据。（3 分）

区别协调组件：

MVP 的核心协调组件是Presenter，由其手动完成 View 和 Model 之间的通信、数据传递和视图更新；

MVVM 的核心协调核心是ViewModel+数据绑定机制，无需手动协调，由数据绑定自动完成 View 和 ViewModel 的状态同步。（4 分）

作业

问题2:

架构模式	数据如何更新?	View 如何与逻辑层交互?
MVP	A	E
MVVM	B	D

问题3

组件交互效率：电商应用的商品列表、购物车、订单页等场景需高频的视图与数据交互，MVVM 通过数据绑定自动同步视图和数据，无需像 MVP 那样由 Presenter 手动更新 View（特性 A），减少了手动协调的代码量，提升了交互响应效率，适配数据实时更新的需求。

开发维护成本：MVVM 实现了 View 与 ViewModel 的彻底解耦，视图开发（如 UI 布局）和业务逻辑开发（如数据处理）可并行进行；后续迭代时，修改 UI 无需改动 ViewModel 代码，修改业务逻辑也无需调整 View，大幅降低了维护和迭代成本，符合项目成本控制要求。

数据流转适配性：电商应用的表单录入（如收货地址）、购物车数量修改、商品筛选等场景，均需要双向的数据流转，MVVM 原生支持双向数据绑定（特性 D），可直接适配该类场景；而 MVP 需通过 Presenter 手动处理双向数据传递，开发效率更低。

更适合采用MVVM架构模式

202505真题（作业）

可选题1:医院知识图谱

公司想要建设一个医院的智能问答系统，建立医院知识图谱，某工认为互联网的关键字检索无法满足大量的数据需求，建议建立知识图谱，采用爬虫技术。

1、【问题1】图1展示了基于知识图谱的医药领域智能问答系统的架构设计方案，请从给出的(a)~(p)中选择，补充完善图1中(1)~(10)空处的内容。（10分）

比如业务层和知识层内部细节比如知识获取知识管理，结构化数据。数据清洗，文本向量化等等。【可选项：(a)网络层、(b)数据层、(c)业务层、(d)知识层、(e)网页数据、(f)结构化数据、(g)数据采集、(h)知识获取、(i)知识清洗、(j)数据清洗、(k)知识管理、(l)实体获取、(m)关系获取、(n)意图识别、(o)语句解析、(p)知识检索。

图1 基于知识图谱的医药领域智能问答系统的架构设计方案

202505真题（作业）

可选题1:医院知识图谱

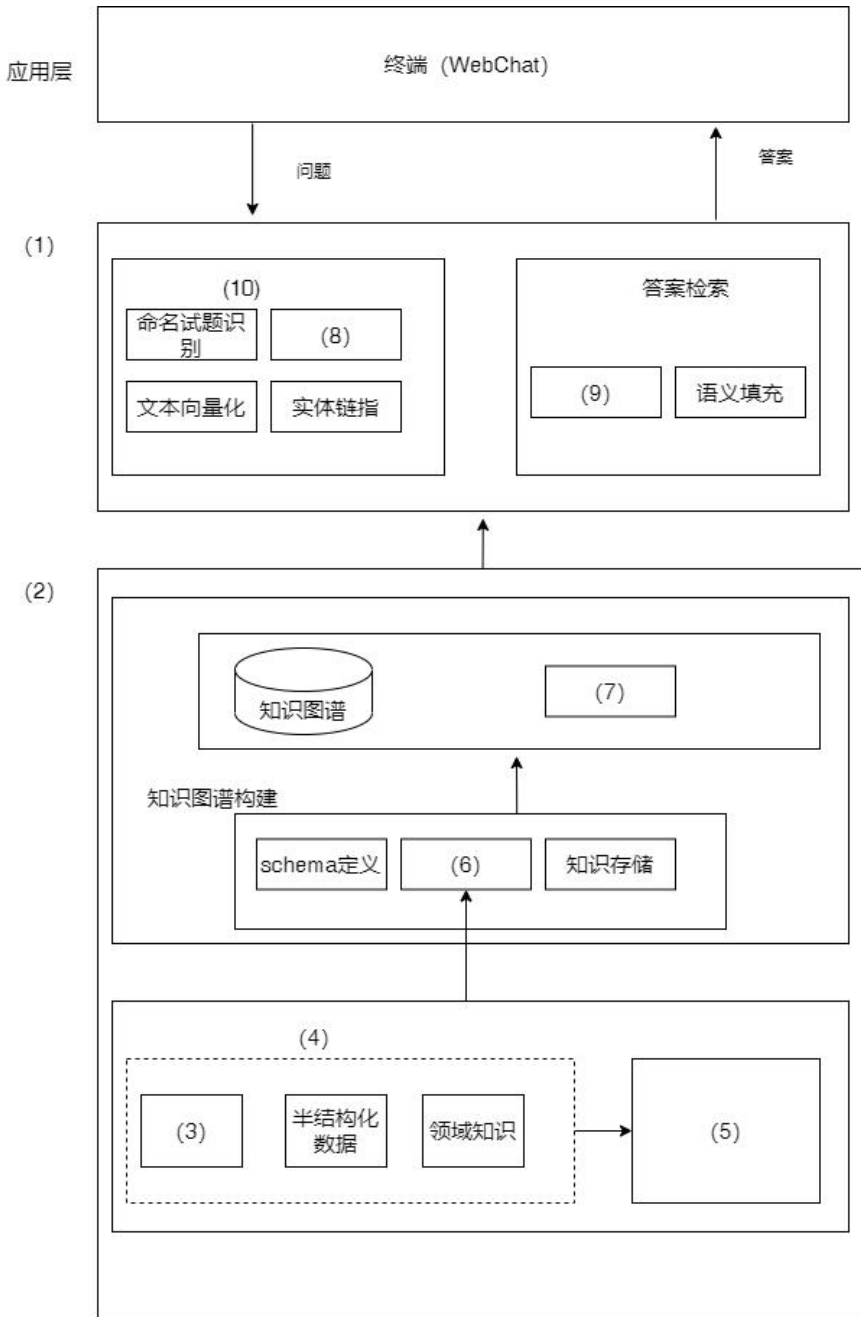
公司想要建设一个医院的智能问答系统，建立医院知识图谱，某工认为互联网的关键字检索无法满足大量的数据需求，建议建立知识图谱，采用爬虫技术。

1、【问题1】图1展示了基于知识图谱的医药领域智能问答系统的架构设计方案，请从给出的(a)~(p)中选择，补充完善图1中(1)~(10)空处的内容。
(10分)

比如业务层和知识层内部细节比如知识获取知识管理，结构化数据。数据清洗，文本向量化等等。

【可选项：(a)网络层、(b)数据层、(c)业务层、(d)知识层、(e)网页数据、(f)结构化数据、(g)数据采集、(h)知识获取、(i)知识清洗、(j)数据清洗、(k)知识管理、(l)实体获取、(m)关系获取、(n)意图识别、(o)语句解析、(p)知识检索。

图1 基于知识图谱的医药领域智能问答系统的架构设计方案



202505真题（作业）

可选题1:医院知识图谱

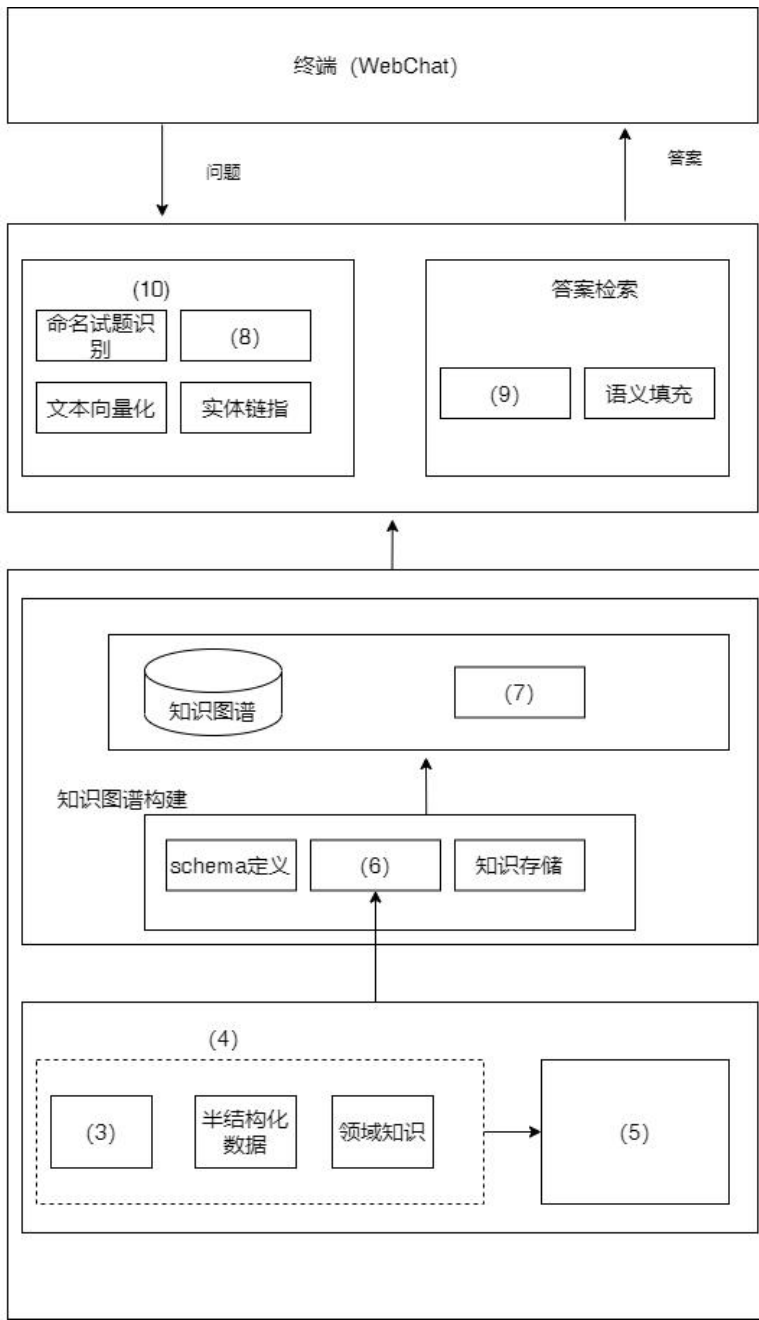
公司想要建设一个医院的智能问答系统，建立医院知识图谱，某工认为互联网的关键字检索无法满足大量的数据需求，建议建立知识图谱，采用爬虫技术。

1、【问题1】图1展示了基于知识图谱的医药领域智能问答系统的架构设计方案，请从给出的(a)~(p)中选择，补充完善图1中(1)~(10)空处的内容。（10分）

比如业务层和知识层内部细节比如知识获取知识管理，结构化数据。数据清洗，文本向量化等等。【可选项：(a)网络层、(b)数据层、(c)业务层、(d)知识层、(e)网页数据、(f)结构化数据、(g)数据采集、(h)知识获取、(i)知识清洗、(j)数据清洗、(k)知识管理、(l)实体获取、(m)关系获取、(n)意图识别、(o)语句解析、(p)知识检索。

图1 基于知识图谱的医药领域智能问答系统的架构设计方案

应用层



【参考答案】：

- (1) : (c) 业务层
- (2) : (b) 数据层
- (3) : (f) 结构化数据
- (4) : (g) 数据采集
- (5) : (j) 数据清洗
- (6) : (h) 知识获取
- (7) : (k) 知识管理
- (8) : (l) 实体获取
- (9) : (p) 知识检索
- (10) : (n) 意图识别

202505综合知识

65.下面关于三层C/S架构的特点描述不正确的是（）。

- A.合理地划分三层的功能，使整个系统的逻辑结构更为清晰，能提高系统的可维护性和可扩展性
- B.B/S架构是一种特殊的两层C/S架构
- C.与两层C/S架构相比，在三层C/S架构中，增加了一个应用服务器
- D.三层C/S架构将应用系统分成表示层、功能层和数据层三个部分

【解析】：

三层C/S 结构如图12-2 (a) 所示。

目前最典型的基于三层C/S 结构的应用模式便是我们最熟悉、较流行的B/S (Browser/Server, 浏览器 / 服务器) 模式，如图12-2 (b) 所示。所以B错误。

【参考答案】： B

66.SOA中（）进一步解耦了服务请求者和服务提供者。

- A.WebService
- B.ESB（企业服务总线）
- C.服务注册表
- D.RMI

【解析】：

ESB 的基本特征和能力包括：描述服务的元数据和服务注册管理；在服务请求者和提供者之间传递数据，以及对这些数据进行转换的能力，并支持由实践中总结出来的一些模式如同步模式、异步模式等；发现、路由、匹配和选择的能力，以支持服务之间的动态交互，解耦服务请求者和服务提供者。高级一些的能力，包括对安全的支持、服务质量保证、可管理性和负载平衡等。

【参考答案】： B

202505综合知识

68.关于架构脆弱性，描述错误的是（）。

- A.分层架构需要穿过多层通信，可能导致性能问题
- B.黑板风格可能没得到解，也没有合适的知识来源继续执行
- C.管道过滤器无法并发
- D.事件驱动中触发事件顺序不可控

【解析】：

管道过滤器支持并发处理，所以C错误。

【参考答案】： C

69.影响软件质量的三组因素是（）。

- A.产品运行、产品修改、产品转移
- B.产品运行、产品修改、产品重构
- C.产品修改、产品转移、产品重构
- D.产品运行、产品重构、产品健壮

【解析】：

从管理角度出发，可以将影响软件质量的因素划分为3 组，分别反映用户在使用软件产品时的3 种不同倾向和观点。这3 组分别是：产品运行、产品修改和产品转移。

【参考答案】： A

202511综合知识

领域架构中，领域分析的核心产出是（8）。

- A.领域模型
- B.领域知识
- C.特定领域
- D.领域样本

（8） 参考答案： A

试题分析

特定领域的架构设计主要分为三个阶段：

- ① 领域分析阶段。这个阶段的主要目标是获得领域模型。领域模型描述领域中系统之间的共同需求，即领域模型所描述的需求为领域需求。
- ② 领域设计阶段。这个阶段的主要目标是获得特定领域的架构，特定领域的架构描述在领域模型中表示的需求的解决方案，它不是单个系统的表示，而是能够适应领域中多个系统的需求的一个高层次的设计。
- 领域实现阶段。这个阶段的主要目标是依据领域模型和特定领域的架构开发和组织可重用信息。这些可重用信息可能是从现有系统中提取得到，也可能需要通过新的开发得到。

202511综合知识

ABSD方法的三个基础不包括（35）。

- (35) A.项目管理
- B.采用架构风格实现质量属性和商业需求
- C.采用软件模板设计软件结构
- D.功能分解

(35) 参考答案：A

试题分析

基于体系结构的软件设计(Architecture-Based Software Design,ABSD)方法是由体系结构驱动的，即指由构成体系结构的商业、质量和功能需求的组合驱动的。使用ABSD方法，设计活动可以从项目总体功能框架明确就开始，这意味着需求抽取和分析还没有完成(甚至远远没有完成),就开始了软件设计。ABSD方法有3个基础。第1个基础是功能的分解。在功能分解中，ABSD方法使用已有的基于模块的内聚和耦合技术。第2个基础是通过选择体系结构（架构）风格来实现质量和商业需求。第3个基础是软件模板的使用，软件模板利用了一些软件系统的结构。

ABSD方法是递归的，且迭代的每一个步骤都是清晰定义的。因此，不管设计是否完成，体系结构总是清晰的，这有助于降低体系结构设计的随意性。

202511综合知识

客户端-服务器架构属于下列哪类架构风格（42）。

- (42) A.层次架构
B.独立构件架构
C.黑板架构
D.数据流架构

(42) 参考答案：A

试题分析

软件体系结构（架构）风格是描述某一特定应用领域中系统组织方式的惯用模式。常见的经典架构风格主要有：

数据流风格：代表有批处理风格、管道过滤器风格。

调用/返回风格：代表有主程序/子程序风格、面向对象风格、层次结构风格、客户端-服务器风格。

独立构件风格：代表有进程通信风格、事件系统体系结构风格

虚拟机风格：代表有：解释器风格、基于规则的系统风格

仓库风格（数据共享/以数据为中心风格）：代表有数据库系统风格、黑板系统风格

超文本系统风格。

客户端-服务器风格通常也会被归属为层次风格。

202511综合知识

一个解释器通常包括完成解释工作的解释引擎、被解释的代码存储区、（51）和记录源代码解释执行进度的数据结构。

- （51） A.解释器引擎的内部状态存储
- B.工作日志存储
- C.保存代码的存储区状态
- D.规则引擎

（51） 参考答案： A

试题分析

一个解释器的核心组件包括：

- ① 解释引擎：这是解释器的“大脑”，负责读取源代码、分析并执行每条指令。
- ② 被解释的代码存储区：用于存放需要解释执行的源代码。
- ③ 记录源代码解释执行进度的数据结构：这通常指的是程序计数器，它跟踪下一条将要被解释执行的指令地址。

解释器引擎的内部状态存储：用于维护解释器在执行过程中的运行环境，例如：存储变量名和其对应的值的符号表，用于函数调用、存储局部变量和控制流程的栈，以及用于动态内存分配的堆。这些内部状态共同记录了程序在执行到某一时刻的“快照”，是解释器能够逐步执行并理解程序逻辑的基础。

202511综合知识

下列哪项属于软件的最佳复用方式（55）。

- (55) A.从其他工程复制粘贴代码
- B.每次重新编写项目代码
- C.使用已验证过的开源软件和工具
- D.复用项目文档

(55) 参考答案：C

试题分析

软件复用是指在开发新的软件系统时，重复使用已有的软件资产，而不是从零开始重新创造。这些资产可以是软件生命周期中的任何产物包括：函数、库、类、模块、框架、架构、设计模式、需求规格、用户手册、测试用例、开发经验、领域知识等。软件复用的核心目标是提高开发效率、降低成本和风险、缩短上市时间，并通过使用经过验证的组件来提升软件质量。

软件复用的最高形式是将经过验证的、可复用的组件作为构建块，通过良好的接口进行组装和集成。选项C使用已验证过的开源软件和工具完美地体现了这一思想，因此是最佳复用方式。

202511综合知识

下列关于进程通信体系结构中消息传递方式的描述，错误的是（57） 。

- (57) A.支持异步传输
B.支持点到点的传递
C.连接件是消息传递机制
D.构件不是独立的

(57) 参考答案： D

试题分析

进程通信体系结构属于独立构件体系结构风格，每种体系结构风格本质上都由各类构件和连接件搭配形成。在进程通信体系结构风格中构件是独立的进程，连接件是进程之间的消息传递机制，可以有直接传递，如点对点传递；以及异步传递，如通过消息队列传递消息等方式。

202511综合知识

家用机器人可根据复杂规则调整行为，如根据环境光线调整清洁模式，实现该功能的核心技术是（58）。

- (58) A.基于规则的系统
B.解释器
C.黑板风格
D.管道-过滤器

(58) 参考答案：A

试题分析

选项A，基于规则的系统使用预定义的规则集来推理和决策，从而根据输入数据动态调整行为。如“如果光线强度低于阈值，则切换至低光照清洁模式”，这跟题干强调的“根据复杂规则调整行为”相符合。

选项B，解释器风格动态解析和执行代码或脚本，但需依赖外部规则定义。因为解释器本身不包含规则逻辑，需配合基于规则的系统或脚本语言使用。解释器风格比较常见的例子是数学表达式求值程序，它通过解析和遍历抽象语法树来执行像“(2+3)*4”这样的字符串指令并输出结果20。题干强调“根据复杂规则调整行为”不是解释器的核心机制。

选项C，黑板风格是多专家系统的分布式协作，通过共享“黑板”数据结构交换信息，逐步解决问题，适用于复杂、模糊问题（如语音识别、图像理解）。题目场景为单一规则匹配，无需黑板架构的分布式协作。

选项D，管道-过滤器模式处理方式是数据流通过管道流经一系列的过滤器处理。过滤器的顺序是固定的，适用于线性数据处理，它强调数据转换而非规则决策，无法直接根据实际情况动态调整来实现不同的行为。

202511综合知识

下列哪项不属于4+1视图的核心组成（59）。

- (59) A.部署视图
- B.进程视图
- C.逻辑视图
- D.执行视图

(59) 参考答案：D

试题分析

4+1架构视图是由Philippe Kruchten提出的一种描述软件架构的经典方法。它通过多个并发的视图来呈现一个系统的不同方面，以满足不同利益相关者（如用户、开发人员、系统管理员等）的关注点。

“4+1”指的是四个主要视图，加上一个关键的统一视角，即场景组成。

逻辑视图：关注系统的功能需求。它描述了系统能向用户提供什么服务。

开发视图：也叫实现视图，关注的是程序代码的实际结构和依赖关系。描述了如何将软件分解为可以由开发团队独立工作的模块或组件，并定义它们之间的层次结构和依赖关系。

进程视图：描述了系统运行时的进程、线程、服务等，以及它们之间如何通信、交互。它关注动态的方面，如系统的运行时行为、并发、同步、性能、可伸缩性等。

物理视图：也叫部署视图，物理视图描述了计算机、服务器、网络设备以及它们之间的连接。也涉及如何将进程和对象部署到这些节点上。也就是系统如何映射到硬件上，它关注的是系统的拓扑结构、部署和运维。

场景：由一小部分关键用例或用户场景构成。这些场景代表了最重要的系统功能或最关键的质量属性需求。可以理解为场景是验证上述视图是否正确的粘合剂和测试用例。比如，我们可以通过“用户下单”这样的场景去测试逻辑视图里面是否有“用户”、“商品”、“订单”、“库存”、“支付”等业务类可以顺利实现下单功能；测试开发视图中实现业务功能的组件如“用户服务”、“订单服务”、“库存服务”、“支付服务”等是否进行了合理划分，组件之间的依赖关系和边界是否清晰；测试进程视图中订单创建、库存扣减、支付处理等操作的执行顺序是否有误，是否存在多个用户同时购买同一种商品这样的并发问题等；以及测试物理视图中数据库、缓存、消息队列等基础设施是否完备，服务间的网络通信是否畅通，明确各个服务部署在哪些服务器上等等。

202511综合知识

软件产品线是一个产品集合，这些产品共享一个公共的、可管理的特征集，这个特征集能满足特定领域的特定需求。软件产品线主要由两部分组成，其中，（68）是领域工程的所有结果的集合，是产品线中产品构造的基础。

- （68）A.核心资源
B.产品集合
C.构件
D.算法和数据结构

（68）参考答案：A

试题分析

软件产品线主要由两部分组成，分别是核心资源和产品集合。核心资源是领域工程的所有结果的集合，是产品线中产品构造的基础。核心资源必定包含产品线中所有产品共享的产品线架构，包括新设计开发的或者通过对现有系统再工程得到的、需要在整个产品线中系统化复用的构件，与构件相关的测试计划、测试实例以及所有设计文档，需求说明书、领域模型、领域范围的定义等。

THANKS

 极客时间 | 训练营